

Aufgabe 1 (lokal getestet)

TCP – bei Initialisierung wird zunächst der three-way-handshake ausgeführt (Wireshark zeichnet drei Pakete auf, der Client sendet SYN an den Server, dieser sendet SYN, ACK zurück und der Client antwortet noch einmal mit ACK).

Beim senden einer Nachricht vom Client an den Server wird eine Nachricht der Länge 0 vom Server an den Client gesendet, also eine Empfangsbestätigung.

Wird der Server geschlossen, sendet dieser eine RST, ACK Nachricht an den Client, signalisiert also, dass die Verbindung getrennt ist. Im noch laufenden Client-Programm kann man zwar Nachrichten eingeben, Wireshark gibt jedoch keinen Hinweis, dass tatsächlich versucht wird diese zu senden (wird der Client geschlossen, sendet dieser eine RST, ACK Nachricht an den Server).

Das Programm unterscheidet klar zwischen einem (oder mehreren) Clients und dem Server.

In Java nutzen *ServerSocket* und *Socket* das TCP-Protokoll (bzw. *StreamSocket* nutzt TCP, wird aber standardmäßig über *Socket* genutzt).

UDP – die Initialisierung erzeugt keinerlei Pakete in Wireshark, da UDP keine dauerhafte Verbindung aufbaut, sondern Pakete „in guter Hoffnung“ an die gegebene Adresse versendet. Sendet cAT (connectAndTalk) an IAT (listenAndTalk), wird dieses Paket ohne Rückantwort übertragen (Wireshark zeichnet nur ein Paket auf).

Wird der IAT geschlossen kann man von cAT weiterhin Nachrichten senden und Wireshark zeichnet das ausgehen dieser auf, allerdings zeichnet Wireshark dann ein ICMP Protokoll auf, dass der angesprochene Port nicht erreichbar ist.

Das Programm initialisiert je einen DatagramSocket für beide Methoden, also keine klare Unterscheidung/Hierarchie zwischen Server und Client wie bei TCP.

In Java nutzen *DatagramSocket* und *DatagramPacket* das UDP-Protokoll.

Aufgabe 2 (lokal getestet)

Code in: *netcat_UDP.java* in Methoden *listenAndTalk* und *connectAndTalk*

Der IAT (listenAndTalk) antwortet auf jede von cAT (connectAndTalk) empfangene Nachricht mit „Echo“. Der IAT reagiert also auf empfangen Nachrichten und cAT kann solche empfangen.

Beobachtung in Wireshark: Sendet cAT ein UDP-Paket, wird dieses von IAT (sofern vorhanden) beantwortet, indem ein UDP-Paket mit Inhalt „Echo“ zurück geschickt wird.

Aufgabe 3 (lokal getestet)

Code in: *netcat_TCP.java* in Methoden *serveClient* und *Client*

Programm funktioniert so, dass der Client eine Nachricht sendet und der Server daraufhin antwortet. So wie implementiert wird immer nur „Nachricht empfangen!“ geantwortet, da kein Code zur Verarbeitung der Client-Eingabe vorhanden, der Server ist so nicht dazu konzipiert als quasi Client für einen Chat-Partner zu fungieren.

Beobachtung in Wireshark:

Alles wie zuvor: three-way-handshake, Paket gesendet und Bestätigung wird gesendet (da Server von sich aus antwortet entsprechend auch zusätzliche Nachricht in die Gegenrichtung und daraufhin Bestätigung des Clients) sowie Nachricht wenn Server oder Client geschlossen werden.

Aufgabe 4 (mit KI-Hilfe, eigene Lösung hat noch nicht funktioniert und mir ging die Zeit aus)

In Datei *TCP_Char_Server.java*

Aufgabe 5

